

Configure incremental models

Learn how to configure and optimize incremental models when developing in dbt.

⚠ SNOWFLAKE COLUMN SIZE CHANGE

Snowflake plans to increase the default column size for string and binary data types in May 2026. [dbt-snowflake](#) versions below v1.10.6 may fail to build certain incremental models when this change is deployed.

> Assess impact and required actions

Incremental models are built as tables in your data warehouse. The first time a model is run, the table is built by transforming *all* rows of source data. On subsequent runs, dbt transforms *only* the rows in your source data that you tell dbt to filter for, inserting them into the target table which is the table that has already been built.

Often, the rows you filter for on an incremental run will be the rows in your source data that have been created or updated since the last time dbt ran. As such, on each dbt run, your model gets built incrementally.

Using an incremental model limits the amount of data that needs to be transformed, vastly reducing the runtime of your transformations. This improves warehouse performance and reduces compute costs.

Configure incremental materializations

Like the other materializations built into dbt, incremental models are defined with `select` statements, with the materialization defined in a config block.

```
{}  
  config(  
    materialized='incremental'  
  )  
{}  
  
select ...
```

To use incremental models, you also need to tell dbt:

- How to filter the rows on an incremental run
- The unique key of the model (if any)

Understand the `is_incremental()` macro

The `is_incremental()` macro powers incremental materializations. It will return `True` if *all* of the following conditions are met:

- The model must already exist as a table in the database
- The `full-refresh` flag *is not* passed
- The running model is configured with `materialized='incremental'`

Note that the SQL in your model needs to be valid whether `is_incremental()` evaluates to `True` or `False`.

Filtering rows on an incremental run

To tell dbt which rows it should transform on an incremental run, wrap valid SQL that filters for these rows in the `is_incremental()` macro.

Often, you'll want to filter for "new" rows, as in, rows that have been created since the last time dbt ran this model. The best way to find the timestamp of the most recent run of this model is by checking the most recent timestamp in your target table. dbt makes it easy to query your target table by using the "`{{ this }}`" variable.

Also common is wanting to capture both new and updated records. For updated records, you'll need to [define a unique key](#) to ensure you don't bring in modified records as duplicates. Your `is_incremental()` code will check for rows created *or modified* since the last time dbt ran this model.

For example, a model that includes a computationally slow transformation on a column can be built incrementally, as follows:

 models/stg_events.sql

```
{{
    config(
        materialized='incremental'
    )
}}

select
    *,
    my_slow_function(my_column)

from {{ ref('app_data_events') }}

{% if is_incremental() %}

    -- this filter will only be applied on an incremental run
    -- (uses >= to include records whose timestamp occurred since the last run
    of this model)
    -- (If event_time is NULL or the table is truncated, the condition will
    always be true and load all records)
where event_time >= (select coalesce(max(event_time), '1900-01-01') from {{
this }} )

{% endif %}
```



OPTIMIZING YOUR INCREMENTAL MODEL

For more complex incremental models that make use of Common Table Expressions (CTEs), you should consider the impact of the position of the `is_incremental()` macro on query performance. In some warehouses, filtering your records early can vastly improve the run time of your query!

About incremental_predicates

`incremental_predicates` is an advanced use of incremental models, where data volume is large enough to justify additional investments in performance. This config accepts a list of any valid SQL expression(s). dbt does not check the syntax of the SQL statements.

This an example of a model configuration in a `yaml` file you might expect to see on Snowflake:

```
models:
  - name: my_incremental_model
    config:
      materialized: incremental
      unique_key: id
      # this will affect how the data is stored on disk, and indexed to limit scans
      cluster_by: ['session_start']
      incremental_strategy: merge
      # this limits the scan of the existing table to the last 7 days of data
      incremental_predicates: ["DBT_INTERNAL_DEST.session_start >
dateadd(day, -7, current_date)"]
      # 'incremental_predicates' accepts a list of SQL statements.
      # 'DBT_INTERNAL_DEST' and 'DBT_INTERNAL_SOURCE' are the standard aliases for the target table and temporary table, respectively, during an incremental run using the merge strategy.
```

Alternatively, here are the same configurations configured within a model file:

```
-- in models/my_incremental_model.sql

{{
  config(
    materialized = 'incremental',
    unique_key = 'id',
    cluster_by = ['session_start'],
    incremental_strategy = 'merge',
    incremental_predicates = [
      "DBT_INTERNAL_DEST.session_start > dateadd(day, -7, current_date)"
    ]
  )
}}
```

...

This will template (in the `dbt.log` file) a `merge` statement like:

```
merge into <existing_table> DBT_INTERNAL_DEST
  from <temp_table_with_new_records> DBT_INTERNAL_SOURCE
  on
    -- unique key
```

```

    DBT_INTERNAL_DEST.id = DBT_INTERNAL_SOURCE.id
    and
    -- custom predicate: limits data scan in the "old" data / existing
table
    DBT_INTERNAL_DEST.session_start > dateadd(day, -7, current_date)
when matched then update ...
when not matched then insert ...

```

Limit the data scan of *upstream* tables within the body of their incremental model SQL, which will limit the amount of "new" data processed/transformed.

```

with large_source_table as (

    select * from {{ ref('large_source_table') }}
    {% if is_incremental() %}
        where session_start >= dateadd(day, -3, current_date)
    {% endif %}

),

...

```

Defining a unique key

Defining the optional [unique_key parameter](#) enables updating existing rows instead of just appending new rows. If new information arrives for an existing `unique_key`, that new information can replace the current information instead of being appended to the table. If a duplicate row arrives, it can be ignored. Refer to [strategy specific configs](#) for more options on managing this update behavior, like choosing only specific columns to update.

If you don't specify a `unique_key`, most adapters will result in `append`-only behavior, which means dbt inserts all rows returned by the model's SQL into the preexisting target table without regard for whether the rows represent duplicates.

The optional `unique_key` parameter specifies a field (or combination of fields) that defines the grain of your model. That is, the field(s) identify a single unique row. You can define `unique_key` in a configuration block at the top of your model, and it can be a single column name or a list of column names.

The `unique_key` should be supplied in your model definition as a string representing a single column or a list of single-quoted column names that can be used together, for

example, `['col1', 'col2', ...]`). Columns used in this way should not contain any nulls, or the incremental model may fail to match rows and generate duplicate rows. Either ensure that each column has no nulls (for example with `coalesce(COLUMN_NAME, 'VALUE_IF_NULL')`) or define a single-column [surrogate key](#) (for example with `dbt_utils.generate_surrogate_key`).

**TIP**

In cases where you need multiple columns in combination to uniquely identify each row, we recommend you pass these columns as a list (`unique_key = ['user_id', 'session_number']`), rather than a string expression (`unique_key = 'concat(user_id, session_number)'`).

By using the first syntax, which is more universal, dbt can ensure that the columns will be templated into your incremental model materialization in a way that's appropriate to your database.

When you pass a list in this way, please ensure that each column does not contain any nulls, or the incremental model run may fail.

Alternatively, you can define a single-column [surrogate key](#), for example with `dbt_utils.generate_surrogate_key` .

When you define a `unique_key` , you'll see this behavior for each row of "new" data returned by your dbt model:

- If the same `unique_key` is present in the "new" and "old" model data, dbt will update/replace the old row with the new row of data. The exact mechanics of how that update/replace takes place will vary depending on your database, [incremental strategy](#), and [strategy specific configs](#).
- If the `unique_key` is *not* present in the "old" data, dbt will insert the entire row into the table.

Please note that if there's a `unique_key` with more than one row in either the existing target table or the new incremental rows, the incremental model may fail depending on your database and [incremental strategy](#). If you're having issues running an incremental model, it's a good idea to double check that the unique key is truly unique in both your

existing database table and your new incremental rows. You can [learn more about surrogate keys here](#).

! INFO

While common incremental strategies, such as `delete+insert + merge`, might use `unique_key`, others don't. For example, the `insert_overwrite` strategy does not use `unique_key`, because it operates on partitions of data rather than individual rows. For more information, see [About incremental_strategy](#).

unique_key example

Consider a model that calculates the number of daily active users (DAUs), based on an event stream. As source data arrives, you will want to recalculate the number of DAUs for both the day that dbt last ran, and any days since then. The model would look as follows:

 models/staging/fct_daily_active_users.sql

```
{{
    config(
        materialized='incremental',
        unique_key='date_day'
    )
}}

select
    date_trunc('day', event_at) as date_day,
    count(distinct user_id) as daily_active_users

from {{ ref('app_data_events') }}

{% if is_incremental() %}

    -- this filter will only be applied on an incremental run
    -- (uses >= to include records arriving later on the same day as the last
    run of this model)
    where date_day >= (select coalesce(max(date_day), '1900-01-01') from {{
this }})

{% endif %}
```

```
group by 1
```

Building this model incrementally without the `unique_key` parameter would result in multiple rows in the target table for a single day – one row for each time dbt runs on that day. Instead, the inclusion of the `unique_key` parameter ensures the existing row is updated instead.

How do I rebuild an incremental model?

If your incremental model logic has changed, the transformations on your new rows of data may diverge from the historical transformations, which are stored in your target table. In this case, you should rebuild your incremental model.

To force dbt to rebuild the entire incremental model from scratch, use the `--full-refresh` flag on the command line. This flag will cause dbt to drop the existing target table in the database before rebuilding it for all-time.

```
$ dbt run --full-refresh --select my_incremental_model+
```

The trailing `+` in the command above will also run all downstream models that depend on `my_incremental_model`. If any of those downstream dependencies are also incremental models, they will be fully refreshed as well.

You can optionally use the [full_refresh_config](#) to set a resource to always or never full-refresh at the project or resource level. If specified as true or false, the `full_refresh` config will take precedence over the presence or absence of the `--full-refresh` flag.

For detailed usage instructions, check out the [dbt run](#) documentation.

What if the columns of my incremental model change?

Incremental models can be configured to include an optional `on_schema_change` parameter to enable additional control when incremental model columns change.

These options enable dbt to continue running incremental models in the presence of schema changes, resulting in fewer `--full-refresh` scenarios and saving query costs.

You can configure the `on_schema_change` setting as follows.

 dbt_project.yml

```
models:
  +on_schema_change: "sync_all_columns"
```

 models/staging/fct_daily_active_users.sql

```
{{
  config(
    materialized='incremental',
    unique_key='date_day',
    on_schema_change='fail'
  )
}}
```

The possible values for `on_schema_change` are:

- `ignore` : Default behavior (see below).
- `fail` : Triggers an error message when the source and target schemas diverge
- `append_new_columns` : Append new columns to the existing table. Note that this setting does *not* remove columns from the existing table that are not present in the new data.
- `sync_all_columns` : Adds any new columns to the existing table, and removes any columns that are now missing. Note that this is *inclusive* of data type changes. On BigQuery, changing column types requires a full table scan; be mindful of the trade-offs when implementing.

Note: None of the `on_schema_change` behaviors backfill values in old records for newly added columns. If you need to populate those values, we recommend running manual updates, or triggering a `--full-refresh`.



`on_schema_change` TRACKS TOP-LEVEL CHANGES

Currently, `on_schema_change` only tracks top-level column changes. It does not track nested column changes. For example, on BigQuery, adding, removing, or modifying a nested column will not trigger a schema change, even if `on_schema_change` is set appropriately.

Default behavior

This is the behavior of `on_schema_change: ignore`, which is set by default.

If you add a column to your incremental model, and execute a `dbt run`, this column will *not* appear in your target table.

If you remove a column from your incremental model and execute a `dbt run`, `dbt run` will fail.

Instead, whenever the logic of your incremental changes, execute a full-refresh run of both your incremental model and any downstream models.

Was this page helpful?

Yes

No

[Privacy policy](#) [Create a GitHub issue](#)

This site is protected by reCAPTCHA and the Google [Privacy Policy](#) and [Terms of Service](#) apply.

 [Edit this page](#)

Last updated on **May 4, 2026**